To some extent, this issue is addressed by transactional support (which we describe in *Transactions*). However, there will still be times where you need to refresh a model object manually. Active Record makes this possible with one line of code—call its `reload` method, and the object's attributes will be refreshed from the database:

```
stock = Market.find_by(ticker: "RUBY")
loop do
  puts "Price = #{stock.price}"
  sleep 60
  stock.reload
end
```

In practice, `reload` is rarely used outside the context of unit tests.

Updating Existing Rows

After such a long discussion of finder methods, you'll be pleased to know that there's not much to say about updating records with Active Record.

If you have an Active Record object (perhaps representing a row from our `orders` table), you can write it to the database by calling its `save` method. If this object had previously been read from the database, this save will update the existing row; otherwise, the save will insert a new row.

If an existing row is updated, Active Record will use its primary key column to match it with the in-memory object. The attributes contained in the Active Record object determine the columns that will be updated—a column will be updated in the database only if its value has been changed. In the following example, all the values in the row for order 123 can be updated in the database table:

```
order = Order.find(123)
order.name = "Fred"
order.save
```